

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНЫМ РАБОТАМ № 6

**«Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Лукошников Дмитрий Викторович

Факультет прикладной информатики

Группа K3242

Направление подготовки 09.03.03 Прикладная информатика

Преподаватели Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими навыками работы с CRUD операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Выполнение:

Практическое задание из файла 6.1(делал после 6.2)

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
 - a. `db.help()`
 - b. `db.help`
 - c. `db.stats()`
4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Переименуйте коллекцию unicorns.
9. Просмотрите статистику коллекции.
10. Удалите коллекцию.
11. Удалите БД learn.

Выполнение:

1-2. Установлен MongoDB для обеих типов систем (32/64 бита) и проверена работоспособность системы запуском клиента mongo.

```
mongosh mongodb://127.0.0.1 × + ▾ - □ ×
Please enter a MongoDB connection string (Default: mongodb://localhost/):

Current Mongosh Log ID: 6a181298f42c61281b3682d0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      8.2.6
Using Mongosh:       2.8.2
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-28T13:00:37.418+03:00: Access control is not enabled for the data base. Read and write access to data and configuration is unrestricted
2026-05-28T13:00:37.419+03:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from , or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----
```

3.Выполнены указанные методы

```
learn> db.help()

Database Class:

getMongo           Returns the current database
connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the
names of all collections in the current database.
getCollectionInfos Returns an array of documents
with collection information, i.e. collection name and options, for the current database.
```

```
learn> db.help

Database Class:

getMongo           Returns the current database
connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the
names of all collections in the current database.
getCollectionInfos Returns an array of documents
with collection information, i.e. collection name and options, for the current database.
```

```
learn> db.stats()
{
  db: 'learn',
  collections: Long('4'),
  views: Long('0'),
  objects: Long('190692'),
  avgObjSize: 33.007614372915484,
  dataSize: 6294288,
  storageSize: 3473408,
  indexes: Long('5'),
  indexSize: 4734976,
  totalSize: 8208384,
  scaleFactor: Long('1'),
  fsUsedSize: 259937312768,
  fsTotalSize: 511164018688,
  ok: 1
}
learn>
```

4. Создана бд learn

```
test> use learn
switched to db learn
```

5. Получил список доступных БД

```
learn> show dbs
admin      40.00 KiB
config    104.00 KiB
learn      7.83 MiB
local      40.00 KiB
learn>
```

6. Создал коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.

```
learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
```

7. Посмотрел список текущих коллекций

```
learn> show collections
habitats
numbers
towns
unicorns
learn> |
```

8.Переименовал коллекцию

```
learn> db.unicorns.renameCollection("mythical_creatures")
{ ok: 1 }
```

9.Просмотрел статистику коллекции.

```
learn> db.mythical_creatures.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
```

10.Удалил коллекцию.

```
learn> db.mythical_creatures.drop()|
```

11.Удалил БД learn.

```
learn> db.dropDatabase()|
```

Практическое задание из файла 6.2

Практическое задание 2.1.1:

1. *Создайте базу данных learn.*

```
test> use learn
switched to db learn
```

2. *Заполните коллекцию единорогов unicorns:*

```
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
| db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
| db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
| db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
| db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
| db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
| db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
| db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
| db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
| db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
| db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a181612f42c61281b3682db') }
}
```

3. *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
| db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1817d1f42c61281b3682de') }
}
```

4. *Проверьте содержимое коллекции с помощью метода find.*

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a181612f42c61281b3682d1'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d2'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d3'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d4'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d5'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d6'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d7'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d8'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
]
```



```
{
  _id: ObjectId('6a181612f42c61281b3682d9'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a181612f42c61281b3682da'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a181612f42c61281b3682db'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a1817d1f42c61281b3682de'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
```

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a1817d1f42c61281b3682de'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d1'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d7'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a181612f42c61281b3682da'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d8'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d4'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d3'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a181612f42c61281b3682d2'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d6'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d9'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a181612f42c61281b3682d2'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a181612f42c61281b3682d2'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a181612f42c61281b3682d2'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```


Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a181612f42c61281b3682d1'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d3'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d4'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d7'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d8'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a181612f42c61281b3682da'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a1817d1f42c61281b3682de'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a1817d1f42c61281b3682de'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a181612f42c61281b3682db'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a181612f42c61281b3682da'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d9'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d8'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d7'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d6'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d5'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {

```

Практическое задание 2.2.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {_id: 0, loves: {$slice: 1}})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: [ 'apple' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Kenny',
    loves: [ 'grape' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Raleigh',
    loves: [ 'apple' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Leia',
    loves: [ 'apple' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Pilot',
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a181612f42c61281b3682db'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```


Практическое задание 2.3.4:

Вывести упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Практическое задание 3.1.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
populatiuon: 6200, last_sensus:
ISODate("2008-01-31"), famous_for:
[""], mayor: {   name: "Jim
Wehrle"  }}

{name: "New York", populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland", populatiuon:
528000, last_sensus:
ISODate("2009-07-20"), famous_for:
["beer", "food"], mayor: {
  name: "Sam Adams", party:
"D"}}
```

```
learn> db.towns.insert(
|   {name: "Punxsutawney",
|   population: 6200,
|   last_sensus: ISODate("2008-01-31"),
|   famous_for: [""],
|   mayor: {
|       name: "Jim Wehrle"
|   }}|
|
```

```
learn> db.towns.insert(
|   {name: "New York",
|     population: 22200000,
|     last_sensus: ISODate("2009-07-31"),
|     famous_for: ["statue of liberty", "food"],
|     mayor: {
|       name: "Michael Bloomberg",
|       party: "I"
|     })
| )
```

```
learn> db.towns.insert(
|   {name: "Portland",
|     population: 528000,
|     last_sensus: ISODate("2009-07-20"),
|     famous_for: ["beer", "food"],
|     mayor: {
|       name: "Sam Adams",
|       party: "D"
|     })
| )
```

2. Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
[
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
learn>
```

3. Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Практическое задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя forEach.

```
learn> showSamchy = function() { return this.gender == "m"; };
[Function: showSamchy]
learn> var cursor = db.unicorns.find({$where: showSamchy}).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj) { print(obj.name) })
Dunx
Horny
learn>
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
2
learn>
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn>
```

Практическое задание 3.3.1:

1. Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a1829f3f42c61281b3682e5')
}
```

В моей версии(8.2.6) монго не поддерживает функцию save, то заменим на аналогичную insertOne

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a1829f3f42c61281b3682e5'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

```
learn> db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a181612f42c61281b3682d6'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```
learn> db.unicorns.update({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a181612f42c61281b3682d8'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Практическое задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.

```
learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, vampires: 1})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
```

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
learn> db.towns.update({name: "Portland"}, {$unset: {"mayor.party": ""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции towns.

```
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a182441f42c61281b3682e4'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

```
learn> db.unicorns.update({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a181612f42c61281b3682da'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

```
learn> db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a181612f42c61281b3682d2'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 3.4.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
  popujatiuon: 6200, last_sensus:
  ISODate("2008-01-31"), famous_for:
  ["phil the groundhog"], mayor: {
  name: "Jim Wehrle" }}

{name: "New York", popujatiuon: 22200000,
  last_sensus: ISODate("2009-07-31"),
```

```
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland", popujatiuon:
528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams", party:
"D"}}
```

```
learn> db.towns.insert(
|   {name: "Punxsutawney",
|     population: 6200,
|     last_sensus: ISODate("2008-01-31"),
|     famous_for: ["phil the groundhog"],
|     mayor: {
|       name: "Jim Wehrle"
|     }}
| {
|   acknowledged: true,
```

```
learn> db.towns.insert(
|   {name: "New York",
|     population: 22200000,
|     last_sensus: ISODate("2009-07-31"),
|     famous_for: ["statue of liberty", "food"],
|     mayor: {
|       name: "Michael Bloomberg",
|       party: "I"
|     }}
| {
|   acknowledged: true,
```

```
learn> db.towns.insert(
|   {name: "Portland",
|     population: 528000,
|     last_sensus: ISODate("2009-07-20"),
|     famous_for: ["beer", "food"],
|     mayor: {
|       name: "Sam Adams",
|       party: "D"
|     }}
| {
|   acknowledged: true,
```

2. Удалите документы с беспартийными мэрами.


```
learn> db.towns.remove({"mayor.party": {$exists: false}})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
learn>
```

3. Проверьте содержание коллекции.

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a18325df42c61281b3682ea'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a18325df42c61281b3682eb'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn>
```

4. Очистите коллекцию.

```
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 2 }
learn>
```

5. Просмотрите список доступных коллекций.

```
learn> show collections
towns
unicorns
learn>
```

Практическое задание 4.1.1:

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.habitats.insert({_id: "rainbow", name: "Rainbow dash", description:
"Colorfull rainbow"})
{ acknowledged: true, insertedIds: { '0': 'rainbow' } }
learn> db.habitats.insert({_id: "mountains", name: "Crystal mountains", descr
iption: "High mountains with crystal cloud"})
{ acknowledged: true, insertedIds: { '0': 'mountains' } }
learn> db.habitats.insert({_id: "forest", name: "Dark Forest", description: "
Forest with magical creatures"})
{ acknowledged: true, insertedIds: { '0': 'forest' } }
learn>
```

2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```
learn> db.unicorns.update({name: 'Horny'}, {$set: {habitat: {$ref: "habitats"
, $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
learn> db.unicorns.update({name: 'Unicrom'}, {$set: {habitat: {$ref: "habitat
s", $id: "mountains"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
learn> db.unicorns.update({name: 'Aurora'}, {$set: {habitat: {$ref: "habitats
", $id: "rainbow"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

3. Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find({habitat: {$exists: true}}, {name: 1, habitat: 1})
[
  {
    _id: ObjectId('6a181612f42c61281b3682d1'),
    name: 'Horny',
    habitat: DBRef('habitats', 'forest')
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d2'),
    name: 'Aurora',
    habitat: DBRef('habitats', 'rainbow')
  },
  {
    _id: ObjectId('6a181612f42c61281b3682d3'),
    name: 'Unicrom',
    habitat: DBRef('habitats', 'mountains')
  }
]
```

Практическое задание 4.2.1:

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
```

Индекс успешно был создан

Практическое задание 4.3.1:

1. Получите информацию о всех индексах коллекции `unicorns`.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

2. Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
```

3. Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

При удалении индекса для идентификации вылезает ошибка

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

```
learn> for (let i = 0; i < 100000; i++) { db.numbers.insert({value: i}) }
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a183ba9f42c61281b396bbf') }
}
learn>
```

2. Выберите последних четыре документа.

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a183ba9f42c61281b396bbf'), value: 99999 },
  { _id: ObjectId('6a183ba9f42c61281b396bbe'), value: 99998 },
  { _id: ObjectId('6a183ba9f42c61281b396bbd'), value: 99997 },
  { _id: ObjectId('6a183ba9f42c61281b396bbc'), value: 99996 }
]
learn>
```

3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)

Без индекса

executionTimeMillis: 359

```

learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 359,
    totalKeysExamined: 0,
    totalDocsExamined: 190676,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 307,
      works: 190682,
      advanced: 4,
      needTime: 190677,
      needYield: 0,
      saveState: 21,
      restoreState: 21,
      isEOF: 1,
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      totalDataSizeSorted: 260,
      usedDisk: false,
      spills: 0,
      spilledRecords: 0,
      spilledBytes: 0,
      spilledDataStorageSize: 0,
      inputStage: {

```

```

    inputStage: {
      stage: 'COLLSCAN',
      nReturned: 190676,
      executionTimeMillisEstimate: 131,
      works: 190677,
      advanced: 190676,
      needTime: 0,
      needYield: 0,
      saveState: 21,
      restoreState: 21,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 190676
    }
  },
  queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
  command: {
    find: 'numbers',
    filter: {},
    sort: { value: -1 },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'DESKTOP-1UH1UE9',
    port: 27017,
    version: '8.2.6',
    gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
learn>

```

4. Создайте индекс для ключа `value`.

```

learn> db.numbers.createIndex({value: 1})
value_1
learn>

```

5. Получите информацию о всех индексах коллекции `numbers`.

```

learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>

```

6. Выполните запрос 2.

```
]
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a183ba9f42c61281b396bbf'), value: 99999 },
  { _id: ObjectId('6a183ba9f42c61281b396bbe'), value: 99998 },
  { _id: ObjectId('6a183ba9f42c61281b396bbd'), value: 99997 },
  { _id: ObjectId('6a183ba9f42c61281b396bbc'), value: 99996 }
]
learn>
```

7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

С индексом

executionTimeMillis: 14

```

learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 14,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,

```



```

    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 0,
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 0,
      keyPattern: { value: 1 },
      indexName: 'value_1',
      isMultiKey: false,
      multiKeyPaths: { value: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'backward',
      indexBounds: { value: [ '[MaxKey, MinKey]' ] },
      keysExamined: 4,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
command: {
  find: 'numbers',
  filter: {},
  sort: { value: -1 },
  limit: 4,
  '$db': 'learn'
},
serverInfo: {
  host: 'DESKTOP-1UH1UE9',
  port: 27017,
  version: '8.2.6',
  gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
learn>

```

8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Запрос с использованием индекса выполняется значительно быстрее (14 мс против 359 мс), так как он сканирует только нужные документы по индексу, в то время как без индекса просматривается вся коллекция из множества документов. Следовательно, запрос с индексом является более эффективным.

Выводы:

В ходе выполнения лабораторной работы были освоены практические навыки работы с СУБД MongoDB. Были изучены основные CRUD-операции: вставка документов с помощью `insert`, выборка данных с использованием `find` с фильтрацией по условиям, сортировка `sort`, ограничение `limit`. Также были рассмотрены операции обновления данных с использованием операторов `$set`, `$inc`, `$push` и удаление документов. Отдельное внимание было уделено созданию ссылок между коллекциями через `DBRef` и настройке индексов для ускорения выполнения запросов. На примере коллекции из 100 000 документов было наглядно показано, что использование индекса позволяет сократить время выполнения запроса с 359 мс до 14 мс, что делает запрос с индексом значительно более эффективным.